Programación reactiva

¿Qué es la programación reactiva?

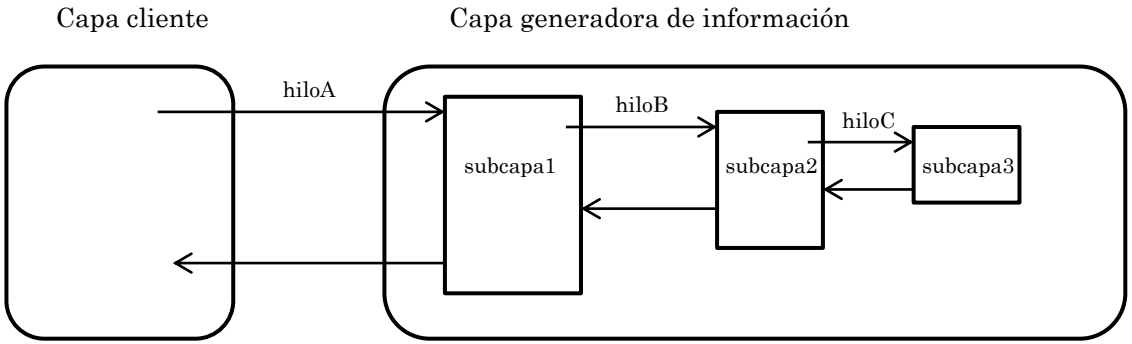
- Modelo de desarrollo para la creación de aplicaciones no bloqueantes

- En una aplicación reactiva, el programa no queda esperando a la llegada de los datos. Los solicita y "reacciona" ante la llegada de los mismos.

- Se trata de aplicaciones de naturaleza asíncrona, en las que el programa puede seguir realizando otras tareas mientras espera la llegada de los datos

Aplicaciones bloqueantes

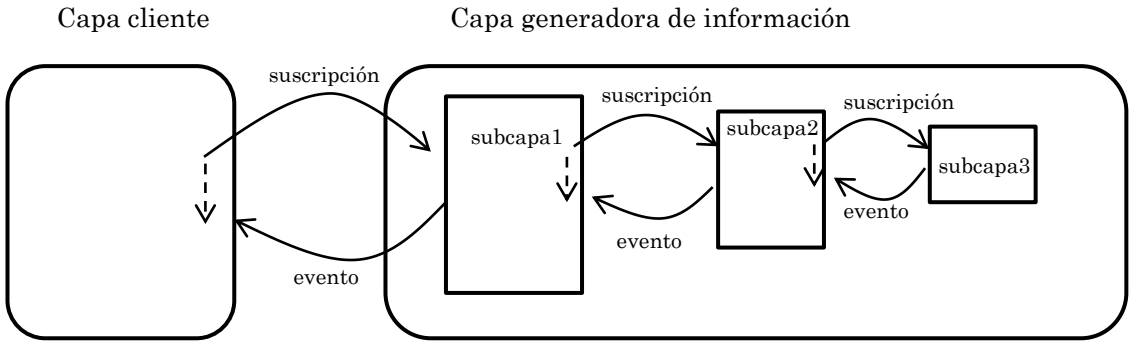
- **Modelo de desarrollo clásico en el que el hilo que hace la llamada a una capa/subcapa queda bloqueado hasta recibir la respuesta:**



- **Pobre rendimiento y elevado consumo de recursos**

Aplicaciones no bloqueantes

- La llamada a la otra capa no bloquea el hilo. Funciona como una suscripción a un evento o flujo de eventos.
- Según van estando disponibles los datos de respuesta, se notifica un evento al cliente que "reacciona" ante el mismo.



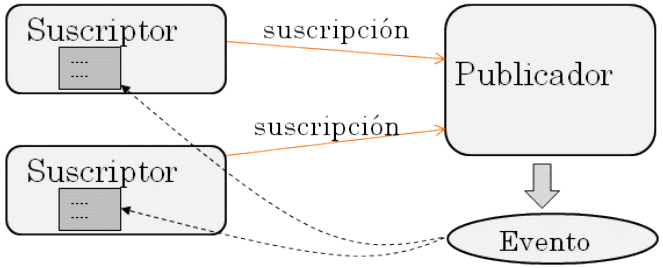
- Los hilos no se bloquean en las llamadas y pueden realizar otras tareas hasta que llegue la notificación del evento.

Patrón publicación-suscripción

- Las aplicaciones reactivas funcionan según los principios del patrón publicación-suscripción

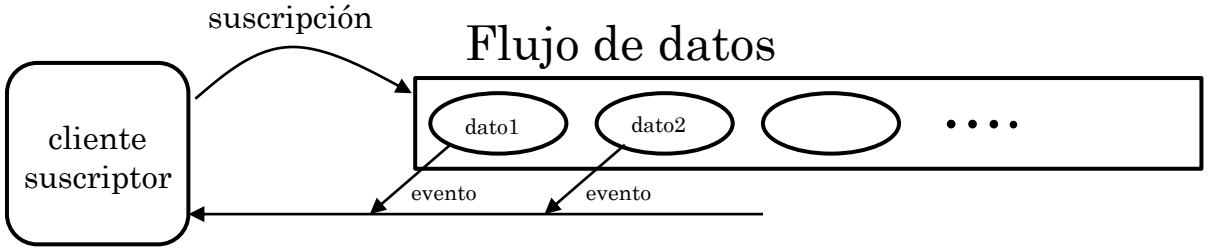
- La capa cliente(suscriptor) se suscribe a un evento en la capa servidor (publicador).

- Cuando se genera un dato de respuesta, el publicador emite un evento hacia el suscriptor. Este reacciona ejecutando un código de respuesta



Flujos de datos

- En aplicaciones reactivas se utiliza el término flujo de datos o stream para referirse a un grupo de datos de respuesta que se emitirán secuencialmente.
- El publicador emite un evento con cada dato del flujo.
- El cliente se suscribe al flujo y reacciona ante cada dato emitido.



APIs para programación reactiva

- Java 9. Nuevo paquete para programación reactiva. Permite la creación del publicador y suscriptores

- Java RX. Combina patrón Observer e iterator, utiliza Observable y Subscriber

- Spring Reactor. Basada en flujos reactivos, creada sobre Java RX, enfocada en la creación de microservicios reactivos.

Casos de éxito

- Netflix. Comenzó utilizando JavaRX para manejar millones de peticiones concurrentes. Utiliza microservicios reactivos para streaming de video

- Twiter(X). Backend asíncrono y no bloqueante para manejar grandes cantidades de tráfico en tiempo real

- Facebook. Utiliza streams reactivos para manejo de eventos a escala global.

Casos de éxito

Spotify. Usa un stack reactivo para procesamiento de eventos de usuario (playlists, reproducciones, recomendaciones). La reproducción de música se basa en modelo no bloqueante.

- **Orange. Implementación de APIs de telecomunicaciones sobre Spring WebFlux, para ofrecer servicios REST reactivos a clientes empresariales.**

Casos de éxito (resumen chat GPT)

📊 Empresas y aplicaciones reactivas (Java WebFlux / RxJava)

Empresa / App	Tecnología reactiva principal	Caso de uso destacado
Netflix	RxJava (creador)	Backend de streaming, Zuul 2 (gateway reactivo), sistemas de recomendaciones.
Alibaba Cloud	Spring WebFlux	APIs de alto rendimiento en su plataforma cloud.
Orange	Spring WebFlux	Exposición de servicios REST para telecomunicaciones.
LINE (mensajería)	Spring WebFlux	Microservicios de mensajería y pagos en tiempo real.
KakaoTalk	Spring WebFlux	Notificaciones y mensajería para millones de usuarios concurrentes.
Trello	RxJava	Sincronización reactiva de tableros y notificaciones en tiempo real.
SoundCloud	RxJava	Streaming de música y app móvil (manejo de UI y caché reactiva).
Pinterest	RxJava	Feed dinámico y scroll infinito en su aplicación Android.
Zalando	Spring WebFlux	Microservicios de e-commerce y APIs públicas.
Expedia	RxJava	Servicios de reservas, integración móvil y pipelines reactivos.
eBay	Spring WebFlux (experimental)	Microservicios de catálogo y APIs internas.